MADRAS INSTITUTE OF TECHNOLOGY

ANNA UNIVERSITY

IOC AD23S02-AI FOUNDATIONS FOR BUSINESS SPECIALIZATION

MINI-PROJECT REPORT

TITLE: AI VIDEO EXPLAINER

MADHESHWARAN G - 2024510005
DURAI GOWSIK G - 2024510018
SARATHI S – 2024510064
PRAVEN KUMAR B.R-2024510309
RAMA VASANTHA PRIYAN S-2024510051

AI VIDEO EXPLAINER

Abstract

In recent years, visual learning has become increasingly important, especially in technical education where concepts can be difficult to grasp through text alone. Many students find it challenging to fully understand engineering topics without clear visual explanations. This project aims to solve that problem by creating an automated system that turns any given topic into a structured explainer video.

The system combines technologies like artificial intelligence, text-to-speech, image retrieval, and video processing to generate complete videos with narration and visuals in just a few minutes. Unlike traditional methods that require manual scripting and editing, this approach automates the entire process. By using the Feynman Technique, the system ensures that explanations are simple, clear, and easy to understand.

Introduction

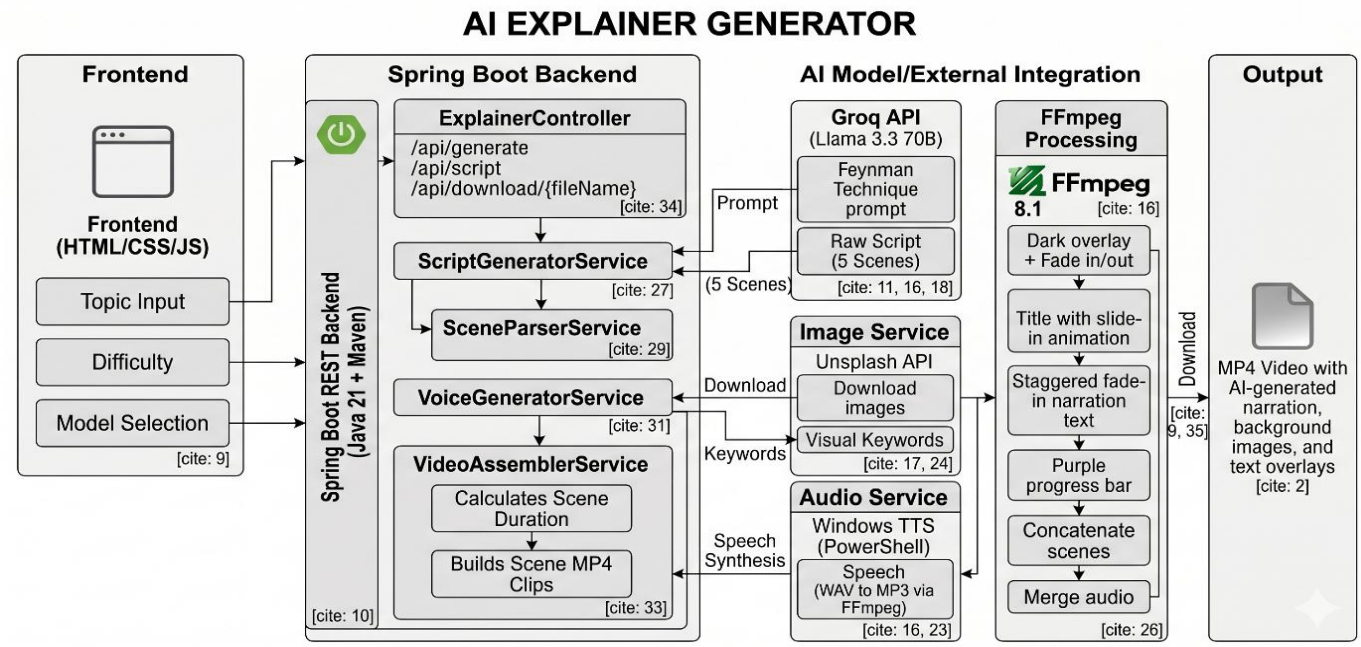
Understanding technical subjects is often difficult for beginners, especially when concepts are explained only through text. Visual learning through videos makes it easier to grasp ideas by combining narration, images, and animations. However, creating such videos manually requires significant time, effort, and technical skills.

This project solves the problem by automating the entire process of explainer video creation. The system generates simple, conversational explanations, similar to how a teacher explains in class. Using the Feynman Technique, it ensures concepts are broken down into clear, easy-to-understand ideas.

Project Gitub Repository Link:

https://github.com/Madheshwaran2006/ai_video_generator_from_topic

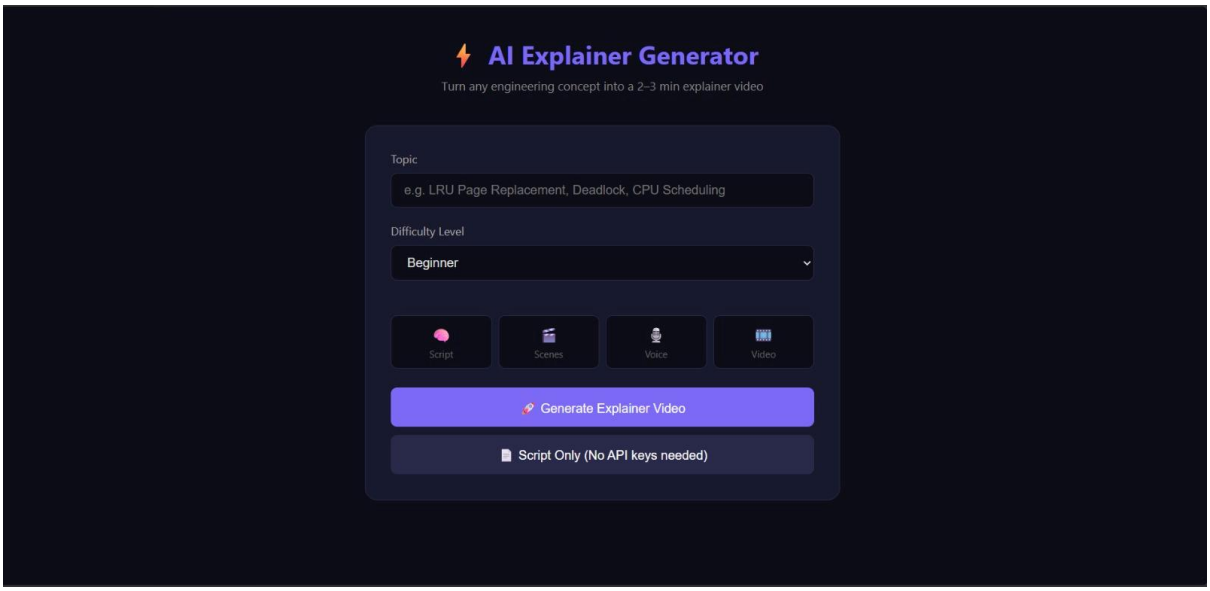
System Architecture



The system architecture is divided into four main layers:

Frontend

The frontend is developed using HTML, CSS, and JavaScript. It provides a simple user interface where the user can enter a topic and select the AI model. The interface is designed to be user-friendly so that even non-technical users can operate the system easily.



Backend (Spring Boot)

The backend is implemented using Spring Boot (Java 21). It acts as the core component of the system and is responsible for handling user requests and coordinating the entire workflow.

It exposes REST APIs such as:

- /api/generate – Generates the complete video
- /api/script – Generates only the script
- /api/download – Downloads the final output

AI and External Services

The system integrates several external tools:

- Groq API (Llama 3.3) – Generates structured scripts
- Windows Text-to-Speech – Converts text into audio
- Unsplash API – Retrieves images
- FFmpeg – Handles video processing

Output

The final output is a downloadable MP4 video that includes:

- Voice narration
- Background images
- Text animations

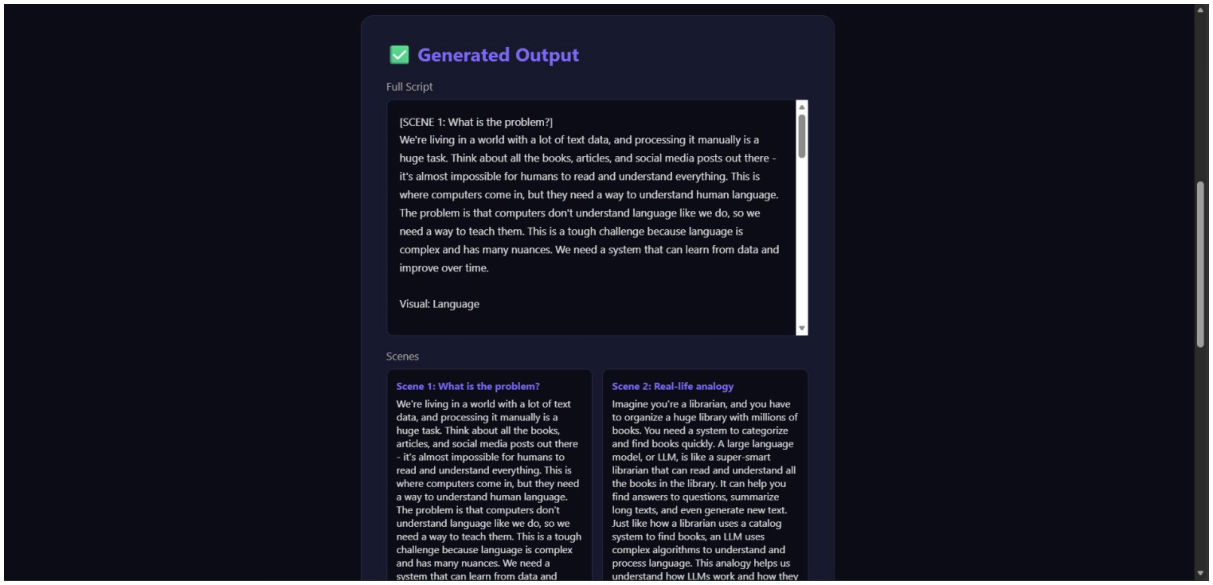
Working Process

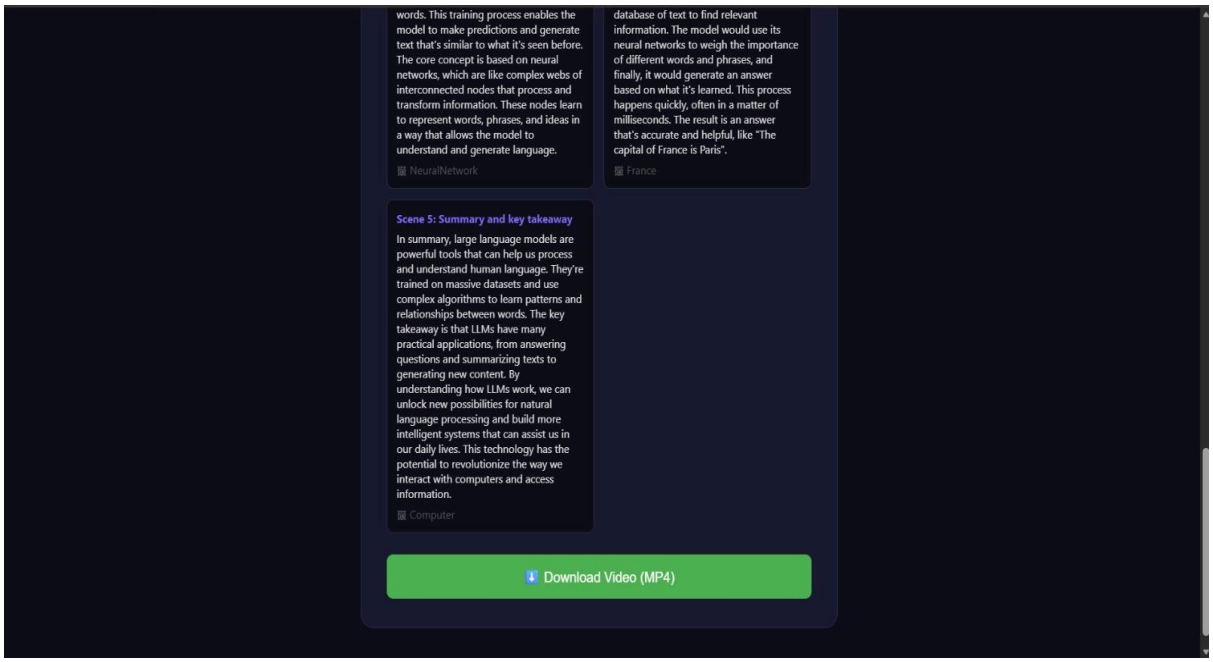
The working of the system is divided into multiple stages.

➢ **Step 1: User Input**

The user enters a topic such as “Deadlock” or “CPU Scheduling” through the interface. This input is sent to the backend for processing.

➢ Step 2: Script Generation





The backend sends a prompt to the AI model using the Groq API. The model generates a structured script consisting of five scenes.

Each scene contains:

- Title
- Narration
- Visual keyword

The narration is written in a conversational style to ensure clarity.

- **Step 3: Scene Parsing**

The generated script is processed using regular expressions. The system extracts each scene and converts it into structured data objects.This step ensures that the content can be used for further processing like audio generation and video creation.

- **Step 4: Audio Generation**

The narration text is cleaned and passed to the Windows Text-to-Speech engine. The output is generated as a WAV file, which is later converted into MP3 format using FFmpeg.

- **Step 5: Image Retrieval**

Each scene contains a visual keyword, which is used to fetch relevant images from the Unsplash API. If the image retrieval fails, the system generates a fallback image.

- **Step 6: Video Assembly**

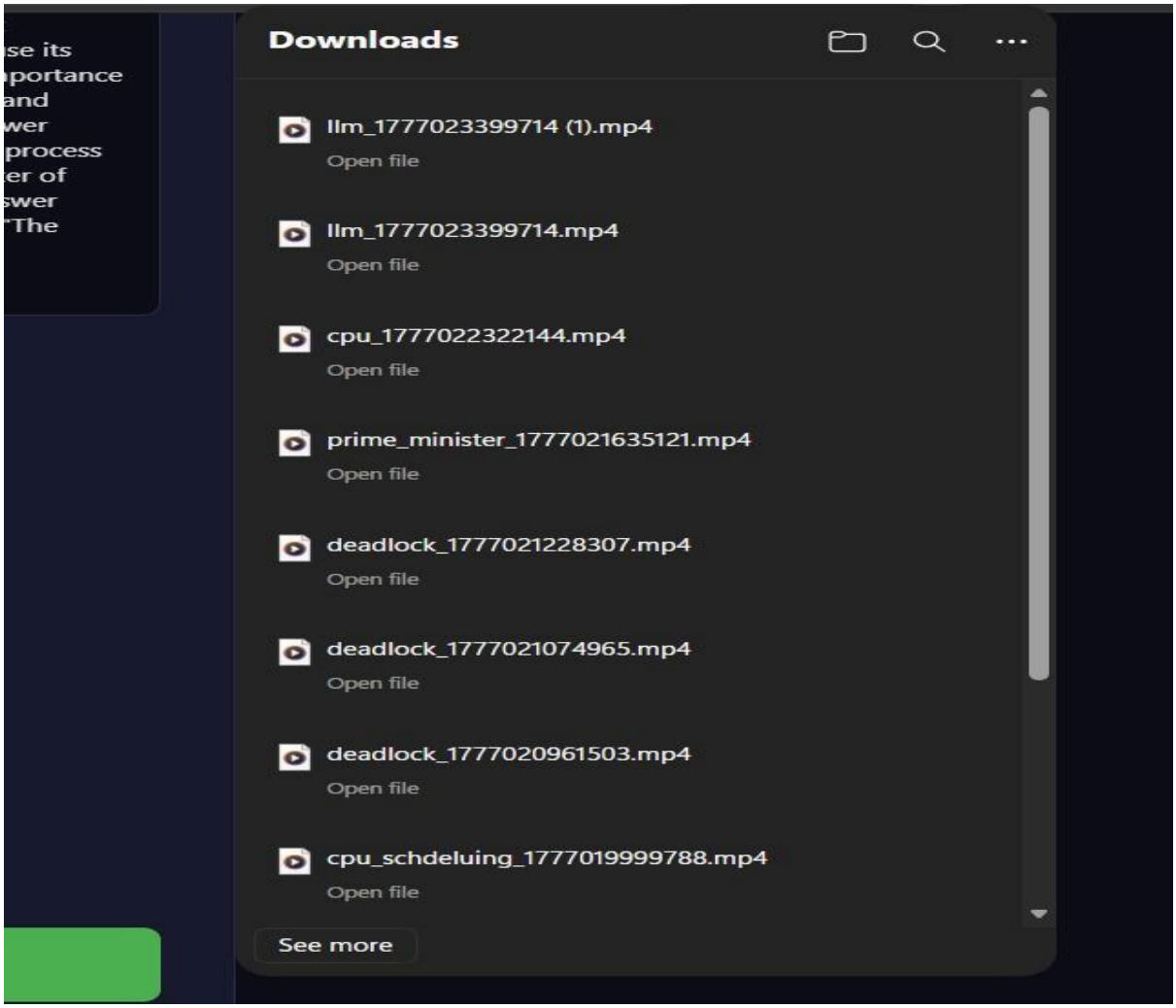
FFmpeg is used to create video clips for each scene. The following features are added:

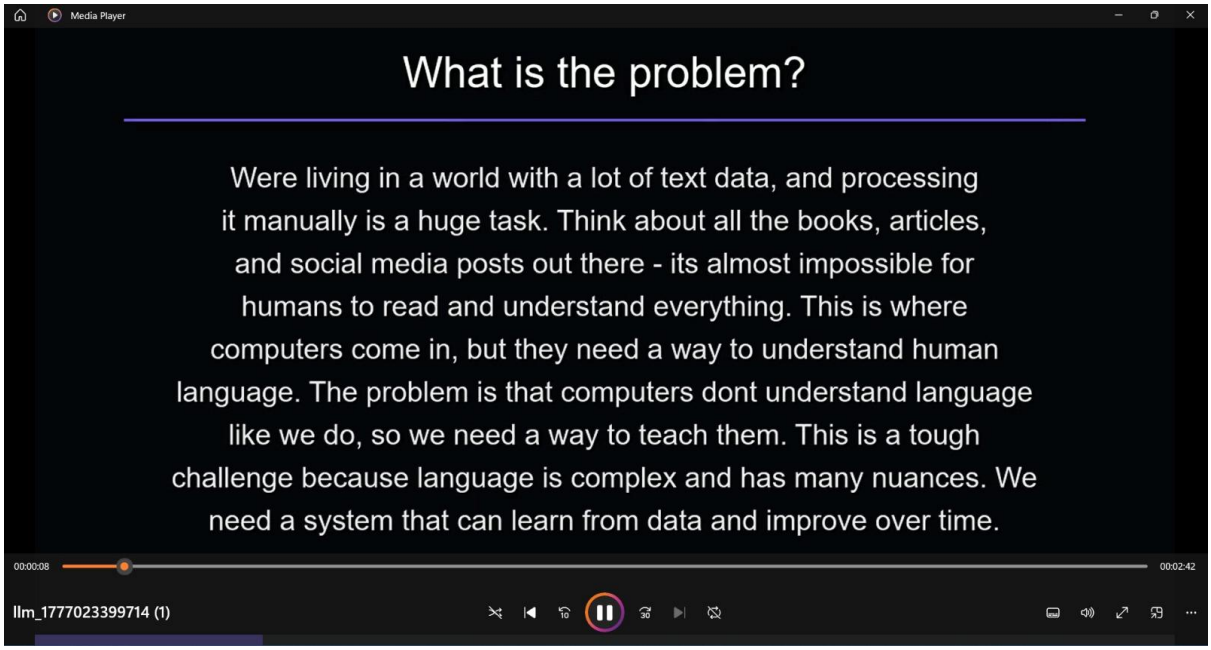
- Background image
- Text overlays
- Fade-in and fade-out animations
- Progress bar

Each scene is converted into a separate video clip**.**

- **Step 7: Final Output**

All scene clips are combined into a single MP4 file. The final video is synchronized with the generated audio and made available for download.


se its
portance
and
wer
process
er of
swer
The
Downloads
llm_1777023399714 (1).mp4
Open file
llm_1777023399714.mp4
Open file
cpu_1777022322144.mp4
Open file
prime_minister_1777021635121.mp4
Open file
deadlock_1777021228307.mp4
Open file
deadlock_1777021074965.mp4
Open file
deadlock_1777020961503.mp4
Open file
cpu_schdeluing_1777019999788.mp4
Open file
See more


Media Player
What is the problem?
Were living in a world with a lot of text data, and processing it manually is a huge task. Think about all the books, articles, and social media posts out there - its almost impossible for humans to read and understand everything. This is where computers come in, but they need a way to understand human language. The problem is that computers dont understand language like we do, so we need a way to teach them. This is a tough challenge because language is complex and has many nuances. We need a system that can learn from data and improve over time.
00:00:08
00:02:42
llm_1777023399714 (1)

Modules Description

The system consists of several modules:

- ScriptGeneratorService
  Responsible for generating scripts using the AI model
- SceneParserService
  Converts raw script into structured scenes
- VoiceGeneratorService
  Handles text-to-speech conversion
- VideoAssemblerService
  Creates video clips and merges them
- ExplainerController
  Manages API requests

Challenges Faced

During development, several challenges were encountered:

- API limitations and quota restrictions
- High cost of premium text-to-speech services
- FFmpeg configuration issues
- Errors in script parsing
- Audio and video synchronization problems

These issues were resolved through debugging, optimization, and tool selection.

8. Challenges Faced and Solutions

Challenge	Solution
OpenAI quota exceeded	Switched to Groq API (free, fast)
ElevenLabs requires subscription	Replaced with Windows built-in TTS
Google TTS requires billing	Used Windows PowerShell Speech Synthesis
FFmpeg font rendering error	Copied Arial font to path without spaces
FFmpeg concat failing	Moved output directory to C:/aiexplainer (no spaces in path)
Scene narration showing 0 chars	Rewrote parser with DOTALL regex and block splitting
Audio not synced with video	Calculated per-scene duration from actual audio length
API keys exposed in properties	Replaced with environment variables

Advantages

- Fully automated system
- Reduces manual effort
- Fast processing time
- Cost-effective solution
- Easy to use

Applications

- Educational platforms
- Online learning systems
- Content creation for YouTube
- Corporate training

Future Enhancements

- Multi-language support
- Background music integration
- Cloud deployment
- Mobile-friendly interface
- Diagram and chart generation

9. Conclusion

This project demonstrates how artificial intelligence and multimedia tools can be combined to create a practical educational solution. By automating the process of video creation, the system saves time and effort while improving the quality of learning.

The project highlights the potential of AI in transforming traditional education into a more interactive and engaging experience.

10. References

- Groq API Documentation — console.groq.com/docs
- Meta Llama Models — ai.meta.com
- FFmpeg Documentation — ffmpeg.org/documentation.html
- Spring Boot — spring.io/projects/spring-boot
- Unsplash API — unsplash.com/developers
- Microsoft Speech Synthesis — Microsoft Docs